

Giáo Án

Giới thiệu

1. Mục tiêu

- Hiểu được React Router DOM là gì, vai trò của nó trong ứng dụng React, và những thay đổi chính trong phiên bản v7.

2. Nội dung chính

- React Router DOM là thư viện định tuyến phổ biến cho React, cho phép điều hướng giữa các trang trong ứng dụng single-page (SPA) mà không cần tải lại trình duyệt.
- Phiên bản v7 (tính đến ngày 16/03/2025) cải tiến API, loại bỏ các prop như `exact` và `end`, tập trung vào khớp route tự động và tính linh hoạt.
- Các tính năng chính: định tuyến tĩnh/động, route lồng nhau, điều hướng programmatic, xử lý lỗi.
- Ví dụ xuyên suốt: Xây dựng ứng dụng Blog với các màn hình Home, Blog List, Blog Detail, Dashboard (với nested routes), và trang 404.

Ví dụ:

- Ý tưởng ban đầu:

Ứng dụng Blog:

- Home: Giới thiệu blog.
- Blog List: Danh sách bài viết.
- Blog Detail: Chi tiết một bài viết với ID.
- Dashboard: Quản lý blog (Posts và Stats).
- NotFound: Trang lỗi khi truy cập sai đường dẫn.

- Cấu trúc thư mục đề xuất:

```
src/
├── components/           # Các component dùng chung
│   ├── NavBar/          # Component điều hướng
│   │   ├── NavBar.jsx
│   │   └── NavBar.css
│   └── ErrorMessage/    # Component hiển thị lỗi (dự phòng, chưa dùng trong ví dụ)
│       ├── ErrorMessage.jsx
│       └── ErrorMessage.css
├── screens/              # Các màn hình chính
│   ├── Home/            # Trang chủ
│   │   └── Home.jsx
│   ├── BlogList/        # Danh sách blog
│   │   └── BlogList.jsx
│   ├── BlogDetail/      # Chi tiết blog
│   │   └── BlogDetail.jsx
│   ├── Dashboard/       # Dashboard (có nested routes)
│   │   ├── Dashboard.jsx
│   │   ├── Posts/       # Quản lý bài viết
│   │   │   └── Posts.jsx
│   │   └── Stats/       # Thống kê
│   │       └── Stats.jsx
│   └── NotFound/        # Trang lỗi 404
│       └── NotFound.jsx
├── App.jsx              # File chính kết nối routes
└── index.css            # CSS global (nếu cần)
```

Hoạt động:

- Trình bày giới thiệu ngắn gọn.
- Đặt câu hỏi: "Tại sao cần React Router DOM để điều hướng giữa các trang trong ứng dụng Blog?"

 Lưu ý cho giảng viên:

- Nhấn mạnh đây là ví dụ xuyên suốt, sẽ phát triển dần qua từng phần.
- Giới thiệu cấu trúc thư mục (`components` cho chung, `screens` cho màn hình) để học viên hình dung cách tổ chức code.

Cài đặt

1. Mục tiêu

- Biết cách cài đặt React Router DOM v7 và cấu hình cơ bản cho ứng dụng Blog

2. Nội dung

- Yêu cầu: Node.js, npm/yarn, dự án React (Vite).
- Câu lệnh cài đặt:

```
npm install react-router-dom
```

- Cấu hình `<BrowserRouter>` trong `App.jsx` để kích hoạt định tuyến.

Ví dụ:

```
// src/App.jsx
import { BrowserRouter } from "react-router-dom";
import Home from "../screens/Home/Home";

function App() {
  return (
    <BrowserRouter>
      <Home />
    </BrowserRouter>
  );
}

export default App;

// src/screens/Home/Home.jsx
function Home() {
  return <h1>Welcome to My Blog</h1>;
}

export default Home;
```

Hoạt động:

- Hướng dẫn học viên cài đặt thư viện và tạo cấu trúc thư mục cơ bản (`screens/Home/Home.jsx`).
- Chạy ứng dụng để kiểm tra trang Home.

 Lưu ý cho giảng viên:

- Đảm bảo học viên kiểm tra phiên bản v7 bằng `npm list react-router-dom` .
- Hướng dẫn tạo thư mục `screens/Home` và file `Home.jsx` để bắt đầu.

Component Routes

1. Mục tiêu

- Hiểu cách sử dụng `<Routes>` để quản lý các route cơ bản trong ứng dụng Blog.

2. Nội dung

- `<Routes>` là container chính để định nghĩa các route, thay thế `<Switch>` trong các phiên bản cũ.
- Tự động chọn route phù hợp nhất, không cần prop `exact`.

Ví dụ:

```
// src/App.jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./screens/Home/Home";
import BlogList from "./screens/BlogList/BlogList";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/blogs" element={<BlogList />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

// src/screens/Home/Home.jsx
function Home() {
  return <h1>Welcome to My Blog</h1>;
}

export default Home;

// src/screens/BlogList/BlogList.jsx
function BlogList() {
  return <h1>Blog List</h1>;
}

export default BlogList;
```

Hoạt động:

- Thực hành: Truy cập `/` và `/blogs` để kiểm tra kết quả.
- Yêu cầu học viên tạo thư mục `screens/BlogList` và file `BlogList.jsx`.

📌 Lưu ý cho giảng viên:

- Nhấn mạnh `<Routes>` tự động xử lý route phù hợp nhất.
- Đảm bảo học viên hiểu cách import từ các thư mục trong `screens`.

Routes và Route

1. Mục tiêu

- Thành thạo cách định nghĩa route tĩnh, động và xử lý lỗi cơ bản trong ứng dụng Blog.

2. Nội dung

- Route tĩnh (/), route động (/blogs/:id) với tham số, route lỗi (*).
- Sử dụng `useParams()` để lấy giá trị tham số từ URL.

Ví dụ:

```
// src/App.jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./screens/Home/Home";
import BlogList from "./screens/BlogList/BlogList";
import BlogDetail from "./screens/BlogDetail/BlogDetail";
import NotFound from "./screens/NotFound/NotFound";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/blogs" element={<BlogList />} />
        <Route path="/blogs/:id" element={<BlogDetail />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

// src/screens/Home/Home.jsx
function Home() {
  return <h1>Welcome to My Blog</h1>;
}

export default Home;

// src/screens/BlogList/BlogList.jsx
function BlogList() {
  return <h1>Blog List</h1>;
}

export default BlogList;

// src/screens/BlogDetail/BlogDetail.jsx
import { useParams } from "react-router-dom";

function BlogDetail() {
  const { id } = useParams();
  return <h1>Blog Detail - ID: {id}</h1>;
}

export default BlogDetail;

// src/screens/NotFound/NotFound.jsx
function NotFound() {
  return <h1>404 - Page Not Found</h1>;
}

export default NotFound;
```

Hoạt động:

- Thực hành: Truy cập `/blogs/1` và `/wrong` để kiểm tra route động và lỗi (20 phút).
- Tạo thư mục `screens/BlogDetail` và `screens/NotFound`, thêm các file tương ứng.

🔴 **Lưu ý cho giảng viên:**

- Giải thích cách `useParams()` lấy `:id` từ URL.
- Nhấn mạnh `path= "*"` là cách xử lý lỗi cơ bản (trang 404).

Link, NavLink và Navigate

1. Mục tiêu

- Sử dụng `<Link>`, `<NavLink>`, `<Navigate>` và `useNavigate` để điều hướng trong ứng dụng Blog.

2. Nội dung

- `<Link>`: Điều hướng cơ bản không tải lại trang.
- `<NavLink>`: Thêm style/class khi route active, phù hợp cho menu.
- `<Navigate>`: Điều hướng programmatic (redirect).
- `useNavigate`: Hook để điều hướng bằng code, hữu ích khi cần chuyển hướng sau một hành động cụ thể (ví dụ: nhấn nút).

Ví dụ:

```
// src/App.jsx
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import NavBar from "./components/NavBar/NavBar";
import Home from "./screens/Home/Home";
import BlogList from "./screens/BlogList/BlogList";
import BlogDetail from "./screens/BlogDetail/BlogDetail";
import NotFound from "./screens/NotFound/NotFound";

function App() {
  return (
    <BrowserRouter>
      <NavBar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/blogs" element={<BlogList />} />
        <Route path="/blogs/:id" element={<BlogDetail />} />
        <Route path="/old-blogs" element={<Navigate to="/blogs" />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

// src/components/NavBar/NavBar.jsx
import { Link, NavLink } from "react-router-dom";
import "./NavBar.css";

function NavBar() {
  return (
    <nav>
      <NavLink to="/" className={{ isActive }} => {isActive ? "active" : ""}>
        Home
      </NavLink>{" "}
    </nav>
  );
}
```

```

    | {" "}
    <NavLink to="/blogs" className={{ { isActive }} => (isActive ? "active" : "")}>
      Blogs
    </NavLink>{" "}
    | <Link to="/blogs/1">Blog #1</Link>
  </nav>
);
}

export default NavBar;

// src/components/NavBar/NavBar.css
.active {
  color: blue;
  font-weight: bold;
}

// src/screens/Home/Home.jsx
function Home() {
  return <h1>Welcome to My Blog</h1>;
}

export default Home;

// src/screens/BlogList/BlogList.jsx
function BlogList() {
  return <h1>Blog List</h1>;
}

export default BlogList;

// src/screens/BlogDetail/BlogDetail.jsx
import { useParams, useNavigate } from "react-router-dom";

function BlogDetail() {
  const { id } = useParams();
  const navigate = useNavigate();

  const handleBack = () => {
    navigate("/blogs");
  };

  return (
    <div>
      <h1>Blog Detail - ID: {id}</h1>
      <button onClick={handleBack}>Back to Blog List</button>
    </div>
  );
}

export default BlogDetail;

// src/screens/NotFound/NotFound.jsx
function NotFound() {
  return <h1>404 - Page Not Found</h1>;
}

export default NotFound;

```

Hoạt động:

- Truy cập `/old-blogs` để xem redirect, kiểm tra style active của `<NavLink>` .
- Kiểm tra style active của `<NavLink>` trong `<NavBar>` khi truy cập `/` và `/blogs` .
- Truy cập `/blogs/1` , nhấn nút "Back to Blog List" để kiểm tra điều hướng bằng `useNavigate` .
- Tạo thư mục `components/NavBar` và thêm file `NavBar.jsx` , `NavBar.css` .

🔑 **Lưu ý cho giảng viên:**

- Nhấn mạnh `<NavBar>` là component dùng chung trong `components` .
- Thêm CSS để học viên thấy hiệu ứng active trực quan.
- Giải thích `/old-blogs` redirect sang `/blogs` .

Nested Routes

1. Mục tiêu

- Hiểu và triển khai route lồng nhau cho Dashboard trong ứng dụng Blog.

2. Nội dung

- Route lồng nhau cho phép hiển thị component con trong component cha.
- Sử dụng `<Outlet>` để render nội dung route con.

Ví dụ:

```
// src/App.jsx
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import NavBar from "./components/NavBar/NavBar";
import Home from "./screens/Home/Home";
import BlogList from "./screens/BlogList/BlogList";
import BlogDetail from "./screens/BlogDetail/BlogDetail";
import Dashboard from "./screens/Dashboard/Dashboard";
import Posts from "./screens/Dashboard/Posts/Posts";
import Stats from "./screens/Dashboard/Stats/Stats";
import NotFound from "./screens/NotFound/NotFound";

function App() {
  return (
    <BrowserRouter>
      <NavBar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/blogs" element={<BlogList />} />
        <Route path="/blogs/:id" element={<BlogDetail />} />
        <Route path="/old-blogs" element={<Navigate to="/blogs" />} />
        <Route path="/dashboard" element={<Dashboard />>
          <Route path="posts" element={<Posts />} />
          <Route path="stats" element={<Stats />} />
        </Route>
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

// src/components/NavBar/NavBar.jsx
import { Link, NavLink } from "react-router-dom";
import "./NavBar.css";
```

```
function NavBar() {
  return (
    <nav>
      <NavLink to="/" className={{ { isActive }} => (isActive ? "active" : "")}>
        Home
      </NavLink>{" "}
      | {" "}
      <NavLink to="/blogs" className={{ { isActive }} => (isActive ? "active" : "")}>
        Blogs
      </NavLink>{" "}
      | <Link to="/blogs/1">Blog #1</Link> | {" "}
      <NavLink to="/dashboard" className={{ { isActive }} => (isActive ? "active" : "")}>
        Dashboard
      </NavLink>
    </nav>
  );
}

export default NavBar;

// src/components/NavBar/NavBar.css
.active {
  color: blue;
  font-weight: bold;
}

// src/screens/Home/Home.jsx
function Home() {
  return <h1>Welcome to My Blog</h1>;
}

export default Home;

// src/screens/BlogList/BlogList.jsx
function BlogList() {
  return <h1>Blog List</h1>;
}

export default BlogList;

// src/screens/BlogDetail/BlogDetail.jsx
import { useParams } from "react-router-dom";

function BlogDetail() {
  const { id } = useParams();
  return <h1>Blog Detail - ID: {id}</h1>;
}

export default BlogDetail;

// src/screens/Dashboard/Dashboard.jsx
import { Link, Outlet } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <nav>
        <Link to="/posts">Posts</Link> | <Link to="/stats">Stats</Link>
```



```
        </nav>
        <Outlet />
    </div>
);
}

export default Dashboard;

// src/screens/Dashboard/Posts/Posts.jsx
function Posts() {
    return <h2>Manage Posts</h2>;
}

export default Posts;

// src/screens/Dashboard/Stats/Stats.jsx
function Stats() {
    return <h2>Blog Stats</h2>;
}

export default Stats;

// src/screens/NotFound/NotFound.jsx
function NotFound() {
    return <h1>404 - Page Not Found</h1>;
}

export default NotFound;
```

Hoạt động:

- Thực hành: Truy cập `/dashboard/posts` và `/dashboard/stats` để kiểm tra nested routes.
- Tạo thư mục `screens/Dashboard` cùng các thư mục con `Posts` và `Stats` , thêm file tương ứng.

📌 Lưu ý cho giảng viên:

- Giải thích cách `/dashboard/posts` render cả `<Dashboard>` và `<Posts>` nhờ `<Outlet>` .
- Nhấn mạnh cấu trúc lồng nhau trong `screens/Dashboard` phù hợp với nested routes.

Hiển thị lỗi

1. Mục tiêu

- Xử lý lỗi định tuyến (404) và cải thiện trải nghiệm người dùng trong ứng dụng Blog.

2. Nội dung

- Sử dụng `path= "*"` để bắt các đường dẫn không khớp.
- Tùy chỉnh trang lỗi với nút quay lại Home.

Ví dụ:

```
// src/App.jsx
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import NavBar from "./components/NavBar/NavBar";
import Home from "./screens/Home/Home";
import BlogList from "./screens/BlogList/BlogList";
import BlogDetail from "./screens/BlogDetail/BlogDetail";
import Dashboard from "./screens/Dashboard/Dashboard";
import Posts from "./screens/Dashboard/Posts/Posts";
```

```
import Stats from './screens/Dashboard/Stats/Stats';
import NotFound from './screens/NotFound/NotFound';

function App() {
  return (
    <BrowserRouter>
      <NavBar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/blogs" element={<BlogList />} />
        <Route path="/blogs/:id" element={<BlogDetail />} />
        <Route path="/old-blogs" element={<Navigate to="/blogs" />} />
        <Route path="/dashboard" element={<Dashboard />}>
          <Route path="posts" element={<Posts />} />
          <Route path="stats" element={<Stats />} />
        </Route>
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

// src/components/NavBar/NavBar.jsx
import { Link, NavLink } from "react-router-dom";
import './NavBar.css';

function NavBar() {
  return (
    <nav>
      <NavLink to="/" className={({ isActive }) => (isActive ? "active" : "")}>
        Home
      </NavLink>{" "}
      | {" "}
      <NavLink to="/blogs" className={({ isActive }) => (isActive ? "active" : "")}>
        Blogs
      </NavLink>{" "}
      | <Link to="/blogs/1">Blog #1</Link> | {" "}
      <NavLink to="/dashboard" className={({ isActive }) => (isActive ? "active" : "")}>
        Dashboard
      </NavLink>
    </nav>
  );
}

export default NavBar;

// src/components/NavBar/NavBar.css
.active {
  color: blue;
  font-weight: bold;
}

// src/screens/Home/Home.jsx
function Home() {
  return <h1>Welcome to My Blog</h1>;
}

export default Home;
```

```
// src/screens/BlogList/BlogList.jsx
function BlogList() {
  return <h1>Blog List</h1>;
}

export default BlogList;

// src/screens/BlogDetail/BlogDetail.jsx
import { useParams } from "react-router-dom";

function BlogDetail() {
  const { id } = useParams();
  return <h1>Blog Detail - ID: {id}</h1>;
}

export default BlogDetail;

// src/screens/Dashboard/Dashboard.jsx
import { Link, Outlet } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <nav>
        <Link to="posts">Posts</Link> | <Link to="stats">Stats</Link>
      </nav>
      <Outlet />
    </div>
  );
}

export default Dashboard;

// src/screens/Dashboard/Posts/Posts.jsx
function Posts() {
  return <h2>Manage Posts</h2>;
}

export default Posts;

// src/screens/Dashboard/Stats/Stats.jsx
function Stats() {
  return <h2>Blog Stats</h2>;
}

export default Stats;

// src/screens/NotFound/NotFound.jsx
import { Link } from "react-router-dom";

function NotFound() {
  return (
    <div>
      <h1>404 - Page Not Found</h1>
      <Link to="/">Back to Home</Link>
    </div>
  );
}
```

```
export default NotFound;
```

Hoạt động:

- Thực hành: Truy cập `/wrong` để xem trang 404 và kiểm tra nút "Back to Home".

📌 Lưu ý cho giảng viên:

- Nhấn mạnh việc tùy chỉnh `<NotFound>` để cải thiện UX.
- Khuyến khích học viên thêm style cho trang 404 (ví dụ: màu đỏ cho tiêu đề) nếu có thời gian.